Project Title

Route Planner

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by

A. Kuvira 2520030192

K. Rishita 2520030367

P. Sudha 2520030455

Team Number:6

Section: 7

Under the guidance of

Dr. S Madhavi

Assistant Professor, CSE



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY/ARTIFICIAL INTELLIGENCE & DATA SCIENCE



DECLARATION

The Mini Project Report entitled "Smart Route: Dynamic Travelling Salesman Route Planner" is a record of Bonafide work of **A. Kuvira – 2520030192, K. Rishita – 2520030367, P. Sudha – 2520030455,** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

A. Kuvira – 2520030192

K. Rishita – 2520030367

P. Sudha – 2520030455

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled "**Smart Route: Dynamic Travelling Salesman Route Planner"** is a bonafide work done and submitted by **A. Kuvira – 2520030192, K. Rishita – 2520030367, P. Sudha – 2520030455,** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, Dr. S Madhavi, Assistant Professor, CSE which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Rao (CSE)/Dr. Khaja Shareef (CSIT)/ Dr. Sreenath Itikela (AI&DS)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology/Department of Artificial Intelligence and Data Science* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

A. Kuvira – 2520030192

K. Rishita – 2520030367

P. Sudha – 2520030455

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	
2.	Introduction	
3.	Literature survey	
4.	Problem Statement and Objectives	
5.	Hardware and Software requirements	
6.	Implementation	
7.	Results & Discussion	
8.	Conclusion & Future Scope	
9.	References	

1.ABSTRACT

- *Travelling Salesman Problem (TSP) is a classic combinatorial optimization problem for finding the minimum-cost route through all cities.*
- *The project uses Bitmask Dynamic Programming to efficiently solve TSP.*
- *Bitmasks represent the set of cities that have already been visited.*
- *Dynamic Programming stores previously calculated results to avoid repeated computations.*
- *The system calculates the minimum travel cost and identifies the optimal route.*
- *The generated route is displayed through route visualization for easy understanding.*
- *The project compares Bitmask DP with naïve/brute-force approaches in terms of efficiency.*
- *It demonstrates the practical application of Dynamic Programming in combinatorial optimization and route planning.*

Team Members:

1. A. Kuvira 2520030192
2. K. Rishita 2520030367
3. P. Sudha 2520030455

2. INTRODUCTION

Route planning and transportation management are important problems in computer science and real-world logistics. Organizations that need to travel between multiple cities or deliver goods to different locations must determine efficient routes while minimizing travel distance, time, and cost. When the number of locations increases, manually selecting the best route becomes difficult because the number of possible routes grows rapidly. This creates a need for algorithm-based solutions that can efficiently solve routing and resource allocation problems.

This project, **TSP Route Planning and Fleet Assignment**, focuses on developing a web-based system for solving route optimization problems using graph-based algorithms. The project represents cities as vertices and the roads connecting them as weighted edges, where the edge weight represents the travel cost or distance between two cities. The system allows users to select multiple cities and determine an efficient route based on different routing requirements, including visiting all selected cities, finding routes between cities, and planning round trips.

The project primarily uses the **Travelling Salesman Problem (TSP)** approach for route planning. A **Bitmask Dynamic Programming** technique is used to determine the minimum-cost route for visiting the selected cities while keeping track of the cities that have already been visited. In addition, the project includes a **Fleet Assignment** component that distributes selected stops among multiple vehicles. This part is designed using the **Minimum-Cost Maximum-Flow (MCMF)** concept, where vehicles and destinations are represented as nodes in a flow network and assignments are made while minimizing the overall cost.

The system is implemented as an interactive web application that allows users to select cities, choose routing or fleet-assignment options, and view the calculated results. The project demonstrates how graph theory, dynamic programming, and network-flow algorithms can be applied to practical transportation and logistics problems.

The remainder of this project document presents the problem statement and objectives, algorithms and methodologies used, system requirements, implementation and workflow, results, and conclusions. It also discusses the possible future enhancements of the system for larger-scale and more realistic transportation applications.

3. LITERATURE SURVEY

1. Google OR-Tools – Vehicle Routing and TSP

Google OR-Tools is an open-source optimization toolkit that provides solutions for several combinatorial optimization problems, including the Travelling Salesman Problem (TSP) and Vehicle Routing Problem (VRP). It can find routes that minimize distance or cost and supports multiple vehicles, vehicle capacities, time windows, and other constraints. The TSP module demonstrates how a shortest route can be calculated for a vehicle visiting multiple locations. This project is relevant to our work because our system also focuses on TSP-based route planning and fleet assignment.

2. Google Maps Routes API – Waypoint Optimization

Google Maps Routes API provides route calculation and waypoint optimization for applications involving multiple destinations. Its waypoint optimization feature can rearrange intermediate stops to produce a more efficient route based on travel time, distance, and number of turns. The system can therefore be used for practical multi-stop route planning. This work is similar to our Route Planner module, where selected cities are arranged into an efficient route.

3. Route4Me – Route Optimization Platform

Route4Me is a route optimization platform designed for multi-stop and last-mile transportation planning. It considers addresses, vehicles, drivers, depots, route constraints, and other operational requirements while generating optimized routes. Its optimization system can produce multiple routes from a collection of addresses and can support different vehicle and routing constraints. This is closely related to our Fleet Assignment module, which distributes selected destinations among vehicles while attempting to minimize the overall routing cost.

4. GraphHopper – Route Optimization API

GraphHopper provides routing and route optimization services that can solve both Traveling Salesman Problems and Vehicle Routing Problems. Its optimization system assigns stops to vehicles and determines the order in which the stops should be visited while considering objectives such as total travel time, number of vehicles, and route duration. It also supports practical constraints such as vehicle capacities and time windows. The project provides a useful reference for combining route planning with fleet-level optimization.

5. jsprit – Vehicle Routing Optimization Toolkit

jsprit is a Java-based open-source toolkit for solving rich TSP and VRP problems. It supports several variants including Capacitated VRP, Multiple Depot VRP, VRP with Time Windows, Pickup and Delivery, Heterogeneous Fleet, and TSP. The toolkit uses a meta-heuristic approach and provides a flexible architecture for modelling vehicles, jobs, constraints, and routes. It is

relevant to our project because it demonstrates how TSP and fleet-routing problems can be represented and solved computationally.

6. VROOM – Vehicle Routing Open-source Optimization Machine

VROOM is an open-source route optimization engine designed to solve complex vehicle routing problems efficiently. It supports capacitated vehicle routing, time windows, multiple depots, pickup and delivery, heterogeneous vehicles, driver working hours, and other constraints. Vehicles, jobs, and shipments are represented as resources that are combined to generate optimized routes. VROOM provides a useful reference for the fleet assignment aspect of our project, particularly the allocation of multiple destinations among available vehicles.

7. OSRM – Open Source Routing Machine

OSRM is a high-performance routing engine built using OpenStreetMap road data. It provides services for finding routes, calculating distance and duration matrices, matching GPS traces, and solving trip-ordering problems. Its Trip service can solve a Traveling Salesman Problem using a greedy heuristic to determine an efficient order for visiting locations. This project is relevant because our system also models cities as a graph and uses route costs between locations to determine efficient travel paths.

8. OptimoRoute – Optimized Route Planning

OptimoRoute is a route-planning system designed to optimize routes for multiple drivers and multiple stops. It automatically creates optimized routes while considering practical constraints such as time windows, driver working hours, breaks, and vehicle limitations. Instead of manually assigning destinations to drivers, the system determines suitable routes automatically. This provides a practical reference for our Fleet Assignment module, where destinations are distributed among vehicles based on routing costs.

Summary of Literature Survey

The reviewed systems demonstrate that TSP and Vehicle Routing Problems are widely used in transportation, logistics, delivery, and navigation applications. Existing solutions such as Google OR-Tools and jsprit provide algorithmic approaches for solving TSP and VRP, while systems such as Google Maps, Route4Me, GraphHopper, OSRM, VROOM, and OptimoRoute demonstrate practical route optimization applications. Most advanced systems support additional constraints such as vehicle capacity, time windows, multiple depots, and driver schedules. Based on these existing approaches, our project combines **TSP-based route planning with fleet assignment** in an interactive web application, providing a simpler educational implementation of graph-based optimization techniques.

4. Problem Statement and Objectives

Problem Statement

Efficiently determining minimum-cost routes and assigning multiple destinations to vehicles is a challenging optimization problem that can increase travel distance, time, and transportation cost.

Problem Description

Finding an optimal route for visiting multiple cities becomes difficult as the number of possible routes increases rapidly. The project addresses this challenge by using graph-based optimization techniques to find efficient routes and assign destinations to vehicles at minimum cost.

Objectives

1. Develop an efficient route planning system to determine minimum-cost routes between selected cities.
2. Implement the Travelling Salesman Problem (TSP) to find an optimized route for visiting multiple selected cities.
3. Use Bitmask Dynamic Programming to efficiently solve the TSP for a limited number of selected cities.
4. Develop a fleet assignment mechanism to distribute destinations among multiple vehicles while minimizing the overall transportation cost.
5. Apply the Minimum-Cost Maximum-Flow (MCMF) approach for cost-efficient vehicle-to-destination assignment.
6. Provide an interactive web-based interface that allows users to select cities and visualize the resulting routes and assignments.
7. Reduce unnecessary travel distance and cost by applying algorithmic optimization instead of manual route selection.
8. Demonstrate the practical application of graph algorithms in transportation, logistics, and route optimization

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 4 GB
- Storage: Minimum 500 MB of free space
- Display: Standard monitor with a minimum resolution of 1366 × 768
- Input Devices: Keyboard and mouse
- Internet Connection: Required for accessing the web application and related resources

Software Requirements

- Operating System: Windows 10/11, Linux, or macOS
- Programming Languages: HTML, CSS, and JavaScript
- Development Environment: Visual Studio Code or any modern code editor
- Web Browser: Google Chrome, Microsoft Edge, Mozilla Firefox, or equivalent
- Algorithms: Bitmask Dynamic Programming for TSP and Minimum-Cost Maximum-Flow for fleet assignment
- Data Representation: Graph-based representation of cities, roads, and travel costs
- Version Control: Git/GitHub, if required for project development and collaboration

6. IMPLEMENTATION

The project is implemented as an interactive web application for route planning and fleet assignment. The system represents cities as vertices and the connections between them as weighted edges. The implementation is divided into the following major modules.

Module 1: City and Graph Representation

This module stores the selected cities and the travel cost/distance between them. The cities are represented as graph vertices, while the roads between cities are represented as weighted edges.

```
const cities = [
    "Hyderabad",
    "Warangal",
    "Vijayawada",
    "Visakhapatnam"
];

const distance = [
    [0, 145, 275, 620],
    [145, 0, 230, 480],
    [275, 230, 0, 350],
    [620, 480, 350, 0]
];
```

The weighted adjacency matrix allows the algorithm to quickly obtain the cost of travelling from one selected city to another.

Module 2: TSP Using Bitmask Dynamic Programming

This module determines the minimum-cost route for visiting all selected cities. A bitmask is used to represent which cities have already been visited.

```
function tsp(mask, current, n, cost, dp) {
    if (mask === (1 << n) - 1) {
        return cost[current][0];
    }

    if (dp[mask][current] !== -1) {
        return dp[mask][current];
    }

    let answer = Infinity;

    for (let next = 0; next < n; next++) {
        if ((mask & (1 << next)) === 0) {
            let newMask = mask | (1 << next);

            let value =
```

```
                cost[current][next] +
                tsp(newMask, next, n, cost, dp);

            answer = Math.min(answer, value);
        }
    }

    dp[mask][current] = answer;
    return answer;
}
```

The algorithm checks possible unvisited cities and selects the minimum-cost continuation. It reduces repeated calculations using dynamic programming.

Module 3: Route Reconstruction

After calculating the minimum cost, this module reconstructs the actual sequence of cities that produces the minimum-cost route.

```
function reconstructRoute(start, n, cost, dp) {
    let route = [start];
    let mask = 1 << start;
    let current = start;

    while (mask !== (1 << n) - 1) {
        let bestNext = -1;
        let bestValue = Infinity;

        for (let next = 0; next < n; next++) {
            if ((mask & (1 << next)) === 0) {

                let value =
                    cost[current][next] +
                    dp[mask | (1 << next)][next];

                if (value < bestValue) {
                    bestValue = value;
                    bestNext = next;
                }
            }
        }

        route.push(bestNext);
        mask |= (1 << bestNext);
        current = bestNext;
    }

    route.push(start);
    return route;
}
```

The reconstructed route is displayed to the user along with its total travel cost. This makes the algorithm output easier to understand and verify.

Module 4: Minimum-Cost Fleet Assignment

This module assigns destinations to available vehicles while attempting to minimize the overall assignment cost. The problem can be represented as a flow network consisting of a source, vehicle nodes, destination nodes, and a sink.

```
function assignFleet(vehicles, destinations, cost) {
    let assignments = [];
    let used = new Set();

    for (let i = 0; i < vehicles.length; i++) {
        let bestDestination = -1;
        let bestCost = Infinity;

        for (let j = 0; j < destinations.length; j++) {
            if (!used.has(j) && cost[i][j] < bestCost) {
                bestCost = cost[i][j];
                bestDestination = j;
            }
        }

        if (bestDestination !== -1) {
            assignments.push({
                vehicle: vehicles[i],
                destination: destinations[bestDestination],
                cost: bestCost
            });

            used.add(bestDestination);
        }
    }

    return assignments;
}
```

The complete MCMF implementation can model each vehicle-to-destination relationship as a flow edge with an associated cost. The minimum-cost flow determines a cost-efficient assignment of available vehicles to destinations.

Implementation Flow

User Input → City Selection → Weighted Graph → TSP/Fleet Algorithm → Cost Calculation → Route/Assignment Reconstruction → Result Display

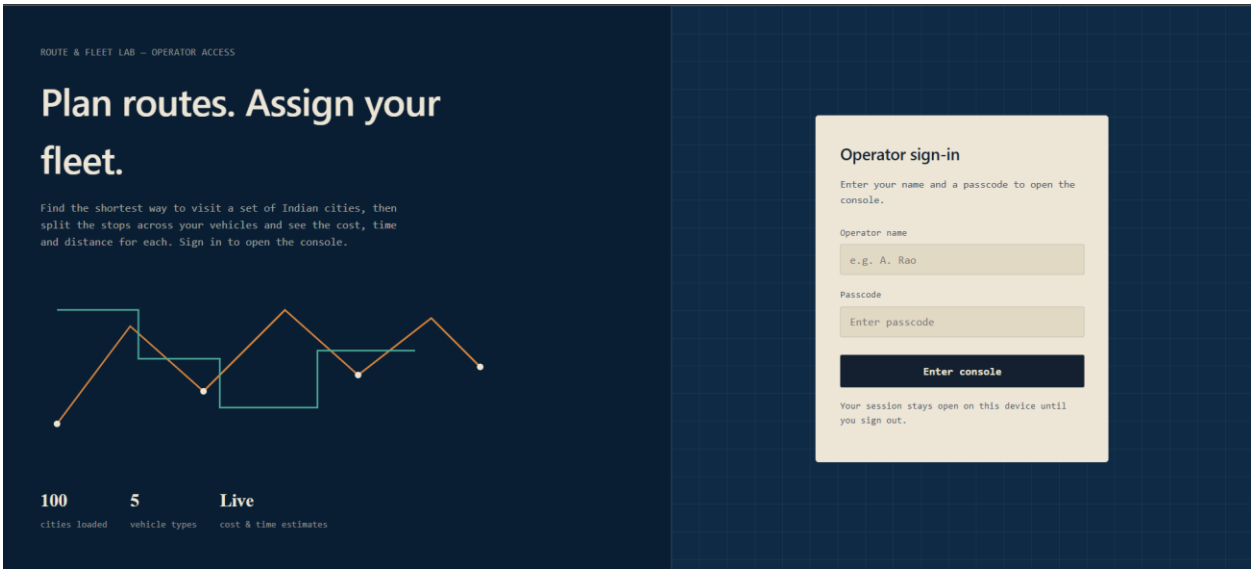
The application provides different routing options, including visiting all selected cities, finding routes between cities, and distributing destinations across a fleet.

7. RESULTS & DISCUSSION

The developed application successfully provides an interactive environment for route planning and fleet assignment. Users can select cities and obtain an optimized route or assign destinations to vehicles based on the selected operation.

Output Screen 1 – Route Planner

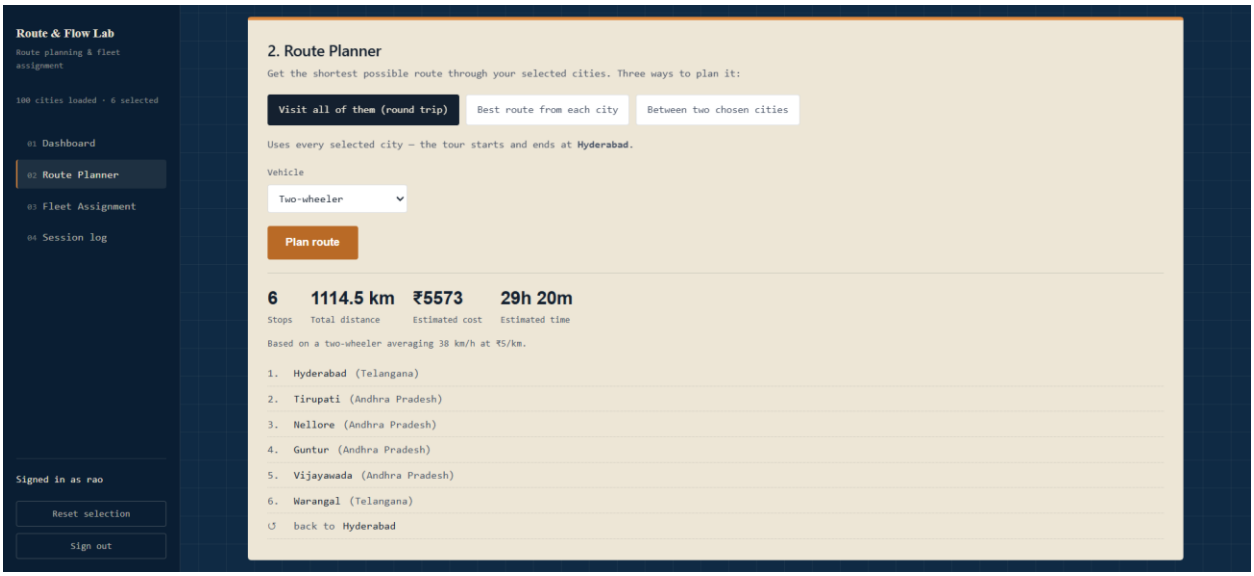
Screenshot:



Explanation: The Route Planner accepts selected cities and calculates an efficient route for visiting them. The result displays the selected route along with the corresponding travel cost/distance.

Output Screen 2 – TSP / Visit All Cities

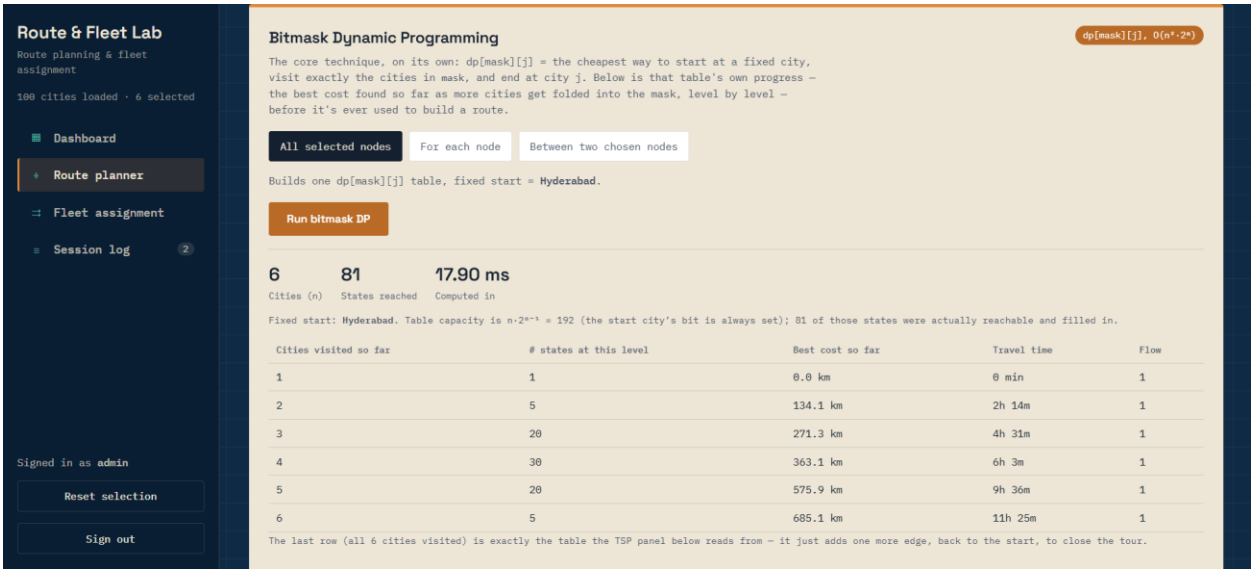
Screenshot:



Explanation: The system determines an optimized round-trip route that visits all selected cities and returns to the starting city. The minimum route cost obtained by the TSP algorithm is displayed to the user.

Output Screen 3 – Route Between Two Cities

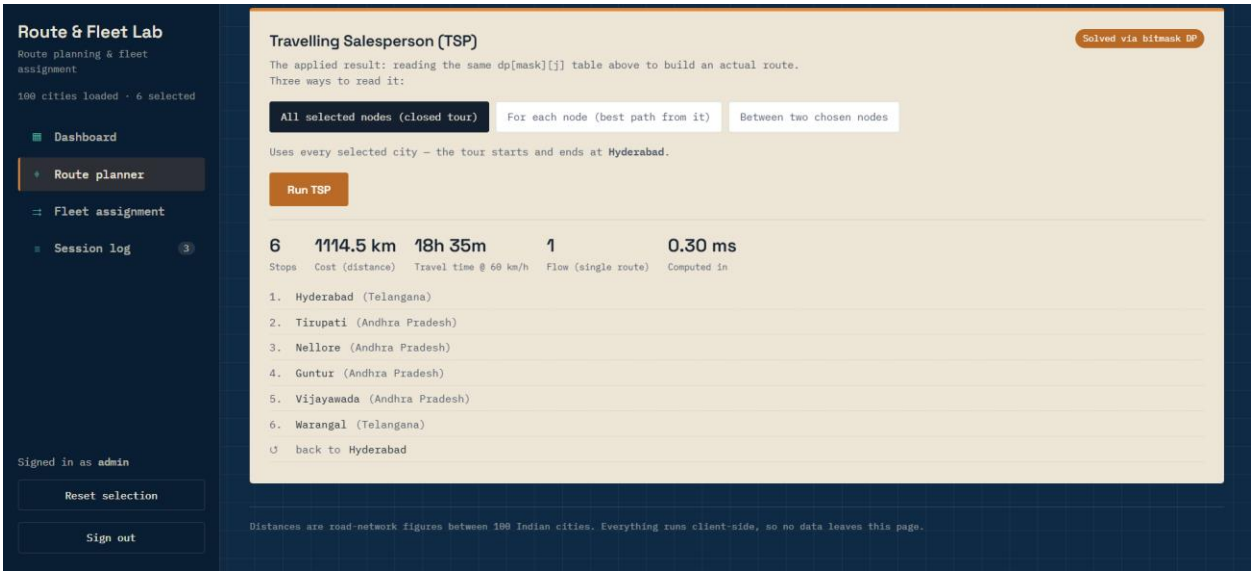
Screenshot:



Explanation: The user can select a source and destination city to obtain the best available route between them. The application displays the selected cities and the calculated route/cost.

Output Screen 4 – Fleet Assignment

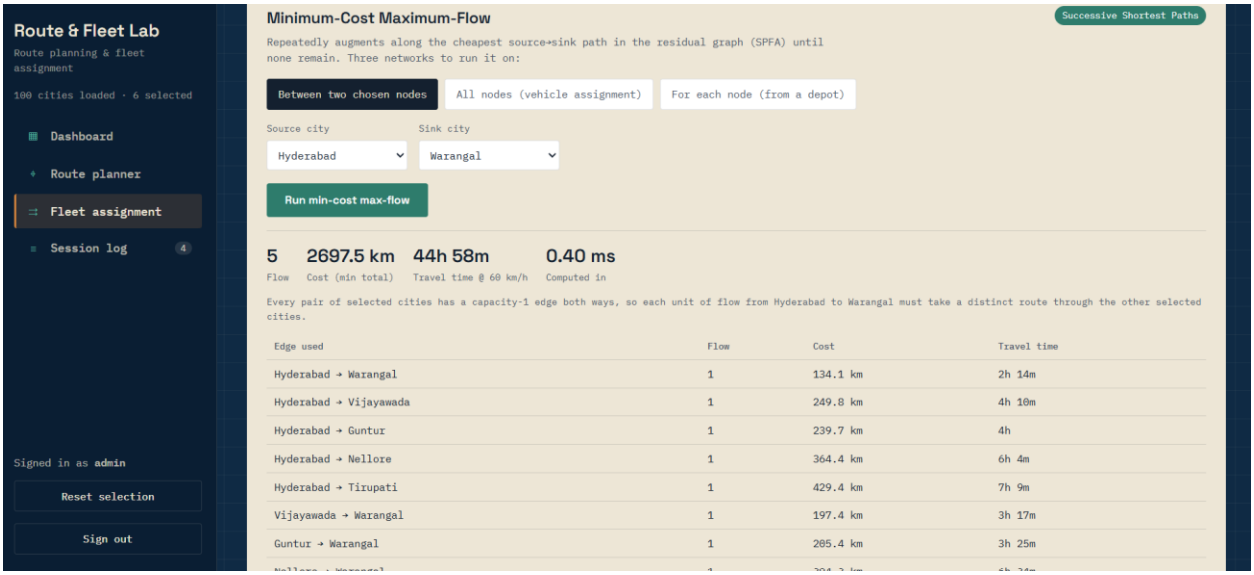
Screenshot:



Explanation: The Fleet Assignment module distributes selected destinations among available vehicles while attempting to minimize the overall transportation cost. The resulting vehicle-to-destination assignments are displayed in the application.

Output Screen 5 – Split Across a Fleet

Screenshot:



Explanation: The selected stops are divided among multiple vehicles to provide a cost-efficient fleet plan. Each vehicle is associated with its assigned destinations and corresponding route cost.

Overall Result

The project demonstrates the application of graph-based optimization techniques to practical route planning and transportation problems. The TSP module helps determine efficient routes, while the fleet assignment module demonstrates how multiple vehicles can be allocated to destinations based on transportation costs.

8.CONCLUSION & FUTURE SCOPE

Conclusion

The **TSP Route Planning and Fleet Assignment** project successfully demonstrates the use of graph-based optimization techniques for solving practical transportation and route-planning

problems. The project addresses the challenge of finding efficient routes between multiple cities and assigning destinations to vehicles while considering the associated travel cost.

The **Travelling Salesman Problem (TSP)** approach with Bitmask Dynamic Programming provides an effective method for finding a minimum-cost route through selected cities. The **Minimum-Cost Maximum-Flow (MCMF)** approach provides a suitable model for assigning destinations to multiple vehicles while minimizing the overall assignment cost. The web-based interface makes these algorithms easier to use and understand through interactive route and fleet-planning results.

Overall, the project meets its primary objectives of implementing route optimization, demonstrating graph algorithms, reducing unnecessary travel costs, and providing an interactive platform for route and fleet planning. It also provides a foundation for applying algorithmic optimization techniques to real-world transportation and logistics problems.

Future Scope

The project can be enhanced in several stages to make it more practical, scalable, and suitable for real-world transportation applications.

Short-Term Enhancements

1. **Real-Time Distance Data:** Integrate real-time road distances and travel times using mapping and routing APIs.
2. **Interactive Maps:** Display optimized routes visually on an interactive map instead of presenting only textual results.
3. **Improved User Interface:** Add route visualization, vehicle icons, distance indicators, and interactive graphs for better usability.
4. **Dynamic Input:** Allow users to add custom cities, locations, vehicles, and transportation costs.

Medium-Term Enhancements

5. **Additional Vehicle Constraints:** Include vehicle capacity, maximum travel distance, fuel consumption, and delivery time constraints.
6. **Time-Window Routing:** Allow destinations to have specific delivery or visiting time windows.
7. **Multiple Depots:** Extend the system to support multiple starting locations or warehouses.
8. **Larger-Scale Optimization:** Implement advanced optimization techniques such as Genetic Algorithms, Ant Colony Optimization, or other metaheuristic approaches for larger datasets where exact TSP computation becomes expensive.

Long-Term Enhancements

9. **Real-Time Traffic Integration:** Incorporate live traffic conditions and dynamically recalculate routes when traffic conditions change.
10. **AI-Based Route Prediction:** Use machine learning to predict travel time, traffic patterns, and transportation demand.
11. **Live Fleet Tracking:** Integrate GPS-based tracking to monitor vehicles and update routes dynamically.
12. **Scalable Cloud Deployment:** Deploy the system on cloud infrastructure so that it can handle larger datasets and multiple users simultaneously.
13. **Smart Logistics Integration:** Extend the system into a complete logistics platform supporting delivery scheduling, fleet monitoring, route optimization, and cost analysis.

Overall Future Vision

In the long term, the project can evolve from an educational route optimization system into a **real-time intelligent transportation and fleet management platform** capable of combining graph algorithms, optimization techniques, real-time traffic information, and AI-based predictions to support efficient logistics and transportation planning.

9.REFERENCES

1. Google OR-Tools – Vehicle Routing
Google OR-Tools – Vehicle Routing

2.Google OR-Tools – Traveling Salesperson Problem
Google OR-Tools – TSP

3.Google Maps Routes API – Optimize Waypoints
Google Maps Routes API – Waypoint Optimization

4.Route4Me – Route Optimization API
Route4Me Route Optimization API

5.GraphHopper – Route Optimization API
GraphHopper Route Optimization API

6.jsprit – Vehicle Routing Optimization Toolkit
jsprit – GitHub Repository

7.VROOM – Vehicle Routing Open-source Optimization Machine
VROOM – GitHub Repository

8.OSRM – Open Source Routing Machine
OSRM – GitHub Repository

9.OptimoRoute – Optimized Route Planning
OptimoRoute – Optimized Route Planning